TALENTTRACK – AN INTELLIGENT PLACEMENT PREPARATION AND CAREER GUIDANCE SYSTEM

A PROJECT REPORT

Submitted by

KIRUTHIKA C	**2303811710422080**
MAHALAKSHMI C	**2303811710422091**
PRADHIKSHA R	**2303811710422117**
PUSHBAJA K	**2303811710422124**

in partial fulfillment for the requirements for the award degree of

Bachelor of Engineering

CSB1401 – MOBILE APPLICATION DEVELOPMENT

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

K.RAMAKRISHNAN COLLEGE OF TECHNOLOGY (AUTONOMOUS)

SAMAYAPURAM-621112

NOV 2026

K.RAMAKRISHNAN COLLEGE OF TECHNOLOGY

(AUTONOMOUS)

SAMAYAPURAM–621112

BONAFIDE CERTIFICATE

The work embodied in the present project report entitled "**TALENTTRACK – AN INTELLIGENT PLACEMENT PREPARATION AND CAREER GUIDANCE SYSTEM**" has been carried out by the students, **KIRUTHIKA C, MAHALAKSHMI C, PRADHIKSHA R, PUSHBAJA K.** The work reported herein is original and we declare that the project is their own work, except where specifically acknowledged, and has not been copied from other sources or been previously submitted for assessment.

Date of Viva Voce: ……

MRS. K. VALLI PRIYADHARSHINI M.E., SUPERVISOR
Assistant Professor,
Department of CSE,
K.Ramakrishnan College of Technology (Autonomous)
Samayapuram–621112

MR. R. RAJAVARMAN M.E.,(Ph.D.,) HEAD OF THE DEPARTMENT
Assistant Professor(Sr.Grade),
Department of CSE,
K.Ramakrishnan College of Technology (Autonomous)
Samayapuram–621112

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ABSTRACT

TalentTrack is an intelligent placement preparation and career guidance system designed to help students prepare effectively for campus placements and job opportunities. The system provides a centralized platform for resume analysis, skill gap identification, personalized study planning, practice activities, and interview preparation. It allows students to upload their resumes and compare them with selected job descriptions to identify existing skills and missing skills required for a target job role. Based on the analysis, a readiness score helps students understand their current preparation level and focus on areas that require improvement. The application also supports placement practice in aptitude, reasoning, verbal ability, technical questions, and coding. The Readiness Engine helps students follow a structured preparation plan based on their skill gaps, while the Interview Planner helps them organize interview-related activities. AI-based analysis can be integrated to process resume and job-description information and provide useful skill-related insights. By combining these features in a single mobile application, TalentTrack aims to make placement preparation more organized, accessible, and personalized, helping students improve their skills, use their preparation time effectively, and build confidence for employment opportunities.

Keywords: TalentTrack, Placement Preparation, Career Guidance, Resume Analysis, Job Description Analysis, Skill Gap Analysis, Readiness Score, Personalized Study Plan, Interview Preparation, Artificial Intelligence.

ACKNOWLEDGEMENT

We thank our **Dr. N. Vasudevan**, Principal, for his valuable suggestions and support during the course of our research work.

We thank **Mr. R. Rajavarman**, Head of the Department, Assistant Professor (Sr. Grade), Department of Computer Science and Engineering, for his valuable suggestions and support during the course of our research work.

We wish to record my deep sense of gratitude and profound thanks to our Guide **Mrs. K. Valli Priyadharshini**, Assistant Professor, Department of Computer Science and Engineering for her keen interest, inspiring guidance, constant encouragement with our work during all stages, to bring this thesis into fruition.

We are extremely indebted to our project coordinator **Mrs. K. Valli Priyadharshini**, Assistant Professor, Department of Computer Science and Engineering, for her valuable suggestions and support during the course of our research work.

We also thank the faculty and non-teaching staff members of the Department of CSE, K.Ramakrishnan College of Technology(Autonomous), Trichy, for their valuable support throughout the course of our research work.

Finally, we thank our parents, friends and our well wishes for their kind support.

SIGNATURE

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	ABSTRACT	**iii**
	LIST OF FIGURES	**vii**
	LIST OF ABBREVIATIONS	**viii**
1	**INTRODUCTION**	**1**
	1.1 DESCRIPTION	1
	1.2 PROBLEM STATEMENT	2
	1.3 PLACEMENT PREPARATION DOMAIN	
	1.3.1 Resume Management and Analysis	3
	1.3.2 Job Description and Skill Gap Analysis	3
	1.3.3 Personalized Placement Preparation	4
	1.4 AIM AND OBJECTIVE	
	1.4.1 Aim	4
	1.4.2 Objectives	4
	1.5 SCOPE OF THE PROJECT	5
		5
2	**LITERATURE SURVEY**	6
3	**EXISTING SYSTEM**	16
	3.1 EXISTING SYSTEM	16
	3.2 DISADVANTAGES	17
4	**PROBLEMS IDENTIFIED**	**18**
5	**PROPOSED SYSTEM**	**19**
	5.1 PROPOSED SYSTEM OVERVIEW	19
	5.2 USER AUTHENTICATION AND PROFILE MANAGEMENT	19
	5.3 RESUME MANAGEMENT AND JOB DESCRIPTION HANDLING	20
	5.4 RESUME ANALYSIS AND SKILL GAP IDENTIFICATION	20

	5.5 PROPOSED SYSTEM ARCHITECTURE	21
	5.6 ADVANTAGES	21
6	**SYSTEM REQUIREMENTS**	**22**
	6.1 HARDWARE REQUIREMENTS	22
	6.2 SOFTWARE REQUIREMENTS	23
7	**SYSTEM IMPLEMENTATIONS**	**25**
	7.1 LIST OF MODULES	25
	7.2 MODULE DESCRIPTION	25
	7.2.1 User Authentication and Account Management Module	25
	7.2.2 Home Dashboard and Navigation	25
	7.2.3 Profile, Education, Skills Management	26
	7.2.4 Resume Management	26
	7.2.5 Job Description Handling	26
	7.2.6 Resume Analysis and Skill Gap Engine	26
	7.2.7 Readiness and Preparation Planning	27
	7.2.8 Practice and Coding Preparation	27
8	**SYSTEM TESTING**	**28**
	8.1 UNIT TESTING	28
	8.2 INTEGRATION TESTING	28
	8.3 SYSTEM TESTING	28
	8.4 PERFORMANCE TESTING	28
	8.5 USABILITY TESTING	28
9	**RESULT AND DISCUSSION**	**29**
10	**CONCLUSION AND FUTURE ENHANCEMENTS**	**31**
	10.1 CONCLUSION	31
	10.2 FUTURE ENHANCEMENTS	32
	APPENDIX A – SOURCE CODE	**33**
	APPENDIX B – SCREENSHOTS	**65**
	REFERENCES	**68**

LIST OF FIGURES

FIGURE NO	FIGURE NAME	PAGE NO
3.1	Existing System	16
5.5	Proposed System	21
B.1	Login and Registration Screen	65
B.2	TalentTrack Dashboard	65
B.3	Resume and Analysis Page	66
B.4	Interview Preparation Page	66
B.5	Guidance And Test Analysis	67
B.6	Mock Interview And Readiness Engine	67

LIST OF ABBREVIATIONS

AI	-	Artificial Intelligence
API	-	Application Programming Interface
JD	-	Job Description
UI	-	User Interface
UX	-	User Experience
MVVM	-	Model–View–ViewModel
PDF	-	Portable Document Format
DOCX	-	Word Open XML Document
SQL	-	Structured Query Language
SDK	-	Software Development Kit
URL	-	Uniform Resource Locator
CGPA	-	Cumulative Grade Point Average

CHAPTER 1

INTRODUCTION

1.1 DESCRIPTION

Campus placement is an important step in the transition from academic study to professional employment. Students are expected to demonstrate technical knowledge, problem-solving ability, communication skills, and readiness for the responsibilities of a job role. Preparing for these requirements can be challenging because recruitment processes vary between organizations and students may not know which skills to prioritize. TalentTrack is an Android application intended to provide a centralized environment for placement preparation and career guidance. It brings together profile management, resume handling, job-role information, skill-gap analysis, practice activities, interview preparation, and progress-oriented preparation. Instead of treating these activities as disconnected tasks, the system organizes them around the student's target role and preparation needs.

A central workflow of TalentTrack is the comparison of resume information with a job description. The analysis can identify skills mentioned in the resume, skills requested by the job description, and possible gaps between the two. The resulting insights can help students decide which topics to revise and which skills to develop. Any readiness score displayed by the application should be based on a documented calculation and presented as an estimate rather than a guarantee of recruitment success. Students often use different platforms for resume creation, aptitude practice, coding exercises, technical revision, and interview preparation

The application is developed for Android using Kotlin and Jetpack Compose. Firebase Authentication supports account access, and Firebase database services may be used to store profile and preparation information according to the implemented configuration. An AI service such as Gemini can be integrated for text analysis when configured. The system is designed to support consistent learning and informed career planning.

1.2 PROBLEM STATEMENT

Students preparing for placements often use separate resources for resume creation, aptitude practice, coding exercises, technical revision, and interview preparation. This fragmented approach makes it difficult to maintain a structured plan and understand overall progress. Students may also be uncertain about how closely their existing skills match the requirements of a specific job description.

A further challenge is the identification of skill gaps. Without comparing their resume and qualifications with a target job role, students may spend time on topics that are not relevant to their immediate goals or overlook important requirements. Students also need a convenient way to practise different recruitment stages and review the areas where they need improvement.

Therefore, an integrated placement preparation and career guidance application is needed to help students manage their profile, compare resume skills with job requirements, identify learning needs, access practice activities, and organize preparation. TalentTrack is designed to address these needs through a single Android application. The system can guide students towards relevant learning topics and practice activities. These may include quantitative aptitude, logical reasoning, verbal ability, programming, technical questions, coding problems, and mock interviews. Where supported by the application, assessment results and preparation activities can be used to review progress. The exact output depends on the functionality implemented and the quality of the information provided. The analysis workflow can compare skills mentioned in the resume with the requirements in a job description and present potential matches and gaps. They may also find it difficult to identify missing skills and decide which topics should be learned first.

Preparation resources are often distributed across multiple platforms. This can make it difficult to track completed activities and maintain a clear learning plan. An integrated application is therefore needed to support resume and job-description comparison, identify potential skill gaps, organize practice activities, and provide career-oriented preparation guidance.

1.3 PLACEMENT PREPARATION DOMAIN

Placement preparation is the process of developing the knowledge and abilities required for recruitment. It includes resume preparation, quantitative aptitude, logical reasoning, verbal ability, programming, technical assessments, project explanation, and HR interview practice. The requirements differ by job role, company, and recruitment stage.

A digital placement preparation system can support this process by making resources easier to access and by helping students structure their learning. Resume and job-description comparison can provide role-specific insights, while practice assessments can help learners identify topics that need further revision. Career guidance connects these activities to longer-term professional goals.

1.3.1 Resume Management and Analysis

Resume management allows students to upload and maintain their resumes within the application workflow. Resume analysis examines the information available in the uploaded document to identify relevant skills and qualifications. The extracted information can then be compared with the requirements of a selected job description.

The usefulness of the analysis depends on the quality of the uploaded document and the accuracy of text extraction. A skill not found in the resume should be treated as a potential missing resume detail rather than proof that the student does not possess that skill.

1.3.2 Job Description and Skill Gap Analysis

A job description contains information about the responsibilities, qualifications, and skills expected for a role. TalentTrack uses the supplied job description as a reference for analysing the resume.The comparison identifies skills mentioned in both the resume and job description, along with requirements that may not be represented in the resume. These results can help the student decide which skills to practise or add to the resume after verifying their actual knowledge.

1.3.3 Personalized Placement Preparation

Personalized preparation involves organising learning and practice activities based on the student's target role and identified preparation needs. For example, a software development role may require programming, problem-solving, and technical interview practice.

TalentTrack's Readiness Engine is intended to connect identified skill gaps with suitable preparation activities. The preparation plan should be based on the features and analysis results actually supported by the application.

1.4 AIM AND OBJECTIVE

1.4.1 Aim

The aim of TalentTrack is to develop an Android-based placement preparation and career guidance system that helps students organize their preparation, understand job-role skill requirements, identify potential skill gaps, and practise for recruitment assessments. The system aims to help students understand their existing skills, identify missing skills based on their target job requirements, and improve their employability through structured practice and interview preparation.

1.4.2 Objectives

- To provide a convenient Android application for placement preparation.
- To support user registration, login, and profile management.
- To provide resume handling and job-description-based skill comparison.
- To identify existing skills and potential missing skills from available text.
- To organize preparation activities according to the student's target role.
- To support aptitude, reasoning, verbal, technical, and coding practice where implemented.
- To support technical and HR interview preparation.
- To help students review preparation activity and assessment results where tracking is implemented.
- To provide career guidance based on job-role requirements and student goals

1.5 SCOPE OF THE PROJECT

The project focuses on students preparing for campus recruitment and entry-level job opportunities. Its scope includes an Android user interface, account access, profile and skill information, resume handling, target job-role information, skill-gap insights, practice activities, and interview preparation. Cloud services support authentication and data management. AI-based analysis may be used for resume and job-description text when the required integration is configured.

The system provides preparation support rather than recruitment decisions. It does not guarantee a job offer, replace professional career counselling, or determine an employer's final selection. The accuracy of any analysis depends on the input data, the text extraction process, and the analysis method. The final report should reflect only those modules implemented and tested in the delivered application.

The project is intended for students who want to organise their preparation for entry-level jobs and campus recruitment. The quality of the analysis depends on the resume content, job description, text extraction, and AI response. TalentTrack does not guarantee placement, replace professional recruitment decisions, or prove a student's actual proficiency solely from the resume.

CHAPTER 2

LITERATURE SURVEY

2.1 A Survey on AI-Based Personalized Career Guidance and Preparation Systems

Author(s): Bhavesh Patil, Saideep Magar, Sarthak Zarekar

Year of Publication: 2026

This paper surveys artificial intelligence-based systems used for personalized career guidance and placement preparation. It discusses how AI and machine learning can help users explore career opportunities by considering their academic background, skills and personal interests. The survey also examines recommendation systems that suggest suitable career paths and learning activities. In addition, it explores the use of AI chatbots and interactive systems for interview practice and feedback.

The paper identifies a limitation in many existing solutions: career discovery, skill development and interview preparation are often handled by separate platforms. An integrated system could make these activities easier to manage. This research is relevant to TalentTrack because the application aims to connect resume analysis, skill-gap identification and placement preparation in a single Android application.

Another important aspect discussed in AI-based career guidance is personalization. Every student has a different academic background, technical skill set, career interest and level of preparation. A personalized system can use these details to provide suggestions that are more relevant than general career advice. Artificial intelligence techniques can support career recommendations by processing information from student profiles and comparing it with the characteristics of different job roles. The paper also highlights the potential of AI-powered chatbots and interview simulation systems. However, separate systems may still be required for resume analysis, learning recommendations and interview practice. The study identifies the lack of an integrated platform as an important research gap.

2.2 A Survey on Artificial Intelligence Based Career Recommendation Systems: Techniques, Architectures, and Research Gaps

Author(s): Ashutosh Shirtode, Prathmesh Utpat, Sanket Wakade
Year of Publication: 2026

This paper examines AI-based career recommendation systems and the techniques used to suggest suitable career paths. It discusses machine learning classifiers, job-matching methods, skill-gap analysis, personalized learning recommendations and fuzzy-logic-based career counselling. These approaches use information about a person's skills, qualifications and career interests to support career-related decisions.

The survey also considers research gaps in existing systems, including fragmented services and the need for more personalized recommendations. It highlights the importance of connecting career guidance with learning support. This study relates to TalentTrack because the proposed application combines resume and job-description analysis with skill-gap identification and preparation activities. Such integration can help students understand their preparation needs more clearly. A useful system should also help users recognize the difference between their current skill set and the requirements of their chosen career. Personalization must also be balanced with transparency, because users need to understand the limitations of automated recommendations. A further consideration is the relationship between career recommendation and skill development. Recommending a job role alone may not be sufficient if the student does not understand the skills required for that role.

Artificial intelligence can help organize large amounts of information and identify relationships between user profiles and career opportunities. This concept is relevant to TalentTrack because resume and job-description comparison can help students recognize skills that may require additional attention. The application can use this information to guide preparation activities, provided the analysis service is properly implemented and tested.

2.3 A Systematic Review of Recommender Systems for Student Academic and Career Guidance

Author(s): Hasna Mahmoud, Mohamed Badouch, Mehdi Boutaounte

Year of Publication: 2025

This study reviews recommender systems designed to support students in academic and career decision-making. It examines research on how recommendation technologies use student information to suggest educational choices and career opportunities. The review considers the methods, evaluation approaches and limitations reported in the selected studies. In education and career guidance, these systems can process student information and provide suggestions related to academic pathways, learning opportunities and possible career choices.

Recommender systems can help students organize information about possible career paths and make decisions based on their profiles and goals. However, the usefulness of recommendations depends on the quality of the information and the methods used to generate them. This work is relevant to TalentTrack because the application aims to use resume details and job requirements to help students identify skills that need further preparation. The effectiveness of a recommendation depends on the information available about the student and the options being recommended. Incomplete profiles or outdated information can produce suggestions that do not match current requirements. Recommendation systems therefore need suitable evaluation methods to determine whether their results are relevant and useful.

This research area relates to TalentTrack because students preparing for placements need guidance that connects their existing skills with the requirements of a target role. TalentTrack aims to support this process through profile information, resume analysis and preparation activities. Mobile applications can provide career-related services in a format that students can access from their phones. The system's recommendations should be reviewed by students rather than accepted as final career decisions. Such recommendations can help students explore opportunities that match their interests and qualifications.

2.4 AI-Based Resume Screening and Job Recommendation System for Students

Author(s): P. Sabitha, C. Sujitha, A. Rubini, S. Nethesha and S. Rajeswari
Year of Publication: 2026

This paper presents an AI-based system for resume screening and job recommendation, with particular attention to students and entry-level job seekers. It discusses the use of Natural Language Processing (NLP) to extract information such as skills, educational qualifications, projects and certifications from resumes. The extracted information is then represented in a structured form for comparison with job requirements. One challenge in automated resume screening is that different candidates may describe similar skills using different words. A basic keyword-matching approach may not recognize all these relationships. The results may also be affected by document formatting, missing information and inaccurate text extraction. Therefore, automated screening should be evaluated using appropriate test documents and expected results.

The proposed approach combines resume parsing with job-matching techniques to estimate how well a candidate's profile fits a job. Automated processing can reduce the effort involved in manually reviewing resumes. However, the quality of the results depends on resume content, text extraction and the matching method. This paper is relevant to TalentTrack's resume-analysis and skill-gap-analysis modules. Resume screening systems attempt to reduce the effort involved in manually examining resumes. A resume may contain information about education, programming languages, projects, certifications and practical experience. Natural Language Processing can be used to extract relevant information from textual documents and organize it into a structured representation. This information can then be compared with job descriptions to identify possible matches.

The concept is relevant to TalentTrack's resume-analysis workflow. By comparing resume content with job requirements, the application aims to help students understand which skills are represented in their resumes and which requirements may need further attention. The analysis must not be treated as proof of practical skill or actual job eligibility.

2.5 Career Match: Job Recommendation System

Author(s): Tamilarasu P., Madheswari S., Manimegalai K. and Muthupriya S.
Year of Publication: 2025

This paper introduces a job recommendation framework intended to improve the matching between candidates and job openings. It discusses challenges such as separate recruitment platforms, inconsistent resume formats and difficulties in identifying suitable candidates for available positions. The proposed framework uses automated resume parsing, semantic representations and explainable ranking to connect candidate profiles with job requirements. Job recommendation systems focus on matching job seekers with suitable employment opportunities. They may consider information such as educational background, skills, experience and job requirements. A recommendation mechanism can compare candidate profiles with job postings and arrange possible matches according to their relevance.

The use of semantic matching can help identify related skills even when resume and job-description wording differs. Explainable ranking can also help users understand which qualifications support a match. These ideas are relevant to TalentTrack because comparing resumes with job descriptions can Semantic matching is particularly useful when the same skill or responsibility is described using different expressions. However, a recommendation system must distinguish between a skill that is explicitly demonstrated in a resume and one that is merely mentioned. It must also consider whether the job description contains clear and current requirements. help students identify relevant skills and areas that may require improvement.

TalentTrack follows a related objective by allowing resume information to be compared with a selected job description. The resulting skill information can support preparation planning. A future improvement could include explaining why a skill is considered relevant to a particular job requirement, making the analysis easier for students to understand.

2.6 Design and Development of an AI-Based Career Navigator, Internship Finder and Document Reminder System

Author(s): Janhvi Devendra Ghule and P. B. Jaipurkar
Year of Publication: 2026

This paper presents the design of an AI-based Android application that integrates career guidance, internship discovery and document management. It discusses the difficulty students face when career planning, internship searching and document handling are spread across different platforms. The proposed application uses student information, including education, skills, interests and career goals, to support career recommendations and skill-gap analysis. Mobile applications can provide career-related services in a format that students can access from their phones. Integrating career guidance, internship discovery and document management may reduce the need to switch between multiple platforms. A student profile can serve as a common source of information for these activities, helping maintain consistency across the application.

The paper emphasizes the value of combining several career-support functions in one mobile application. It is relevant to TalentTrack because TalentTrack also aims to provide a unified environment for placement preparation. The study supports the idea of integrating profile information, resume management and career preparation activities to make the process more organized. Document management is also important in career preparation because students may need to organize resumes, certificates and other supporting documents. A mobile-based approach can make these resources easier to access. However, the application should protect personal information and provide clear feedback when uploads or external services fail.

This study's general theme is relevant to TalentTrack because the project is intended to provide placement-related features in a single Android application. TalentTrack focuses on resume handling, job-description comparison and preparation support. Its value depends on reliable integration and a user interface that is easy for students to understand.

2.7 Job Recommender Systems: A Systematic Literature Review, Applications, Open Issues, and Challenges

Author(s): Duygu Çelik Ertuğrul and Selin Bitirim
Year of Publication: 2025

This review examines job recommender systems that connect job seekers with suitable job openings. It discusses approaches such as content-based filtering, collaborative filtering and other recommendation techniques. These systems may use resume information, educational qualifications, work experience, skills and job preferences to identify potential matches. This systematic review examines job recommender systems and the approaches used to match candidates with job postings. The authors reviewed research published between 2010 and 2023, including survey articles and individual research papers. They classified methods according to recommendation techniques, applications, development tools and evaluation metrics. The review discusses content-based, collaborative, hybrid and knowledge-based filtering, among other approaches.

The review explains that resume information must be extracted and represented appropriately before it can be compared with job postings. It also discusses limitations in existing approaches, including evaluation challenges and the need for better recommendation methods. This work is relevant to TalentTrack because job-description comparison is an important part of identifying the skills required for a target role. The review also highlights challenges such as recommendation quality, bias, scalability, privacy and the difficulty of evaluating different methods consistently. Systems may use similarity calculations, machine learning and knowledge-based methods to compare candidates with job requirements. The choice of technique depends on the available data and the goal of the system.

This literature is relevant to TalentTrack because comparing resume information with job descriptions is closely related to candidate-job matching. It also highlights why the application's results should be evaluated carefully. A readiness score or skill match should be explained as an estimate based on available information, not as a definitive measure of a student's ability or chance of getting a job.

2.8 Navigating Techniques in Job Recommender Systems on Internship Profile Matching: A Systematic Review

Author(s):Flordeliza P. Poncio

Year of Publication: 2024

This systematic review investigates techniques used to match internship opportunities with candidate profiles. It examines how researchers collect information about candidates, which classification algorithms and ranking metrics are used, and how the effectiveness of recommendations is evaluated. Internship recommendation systems aim to help students identify opportunities that are relevant to their academic background and skills. Candidate profiles may include educational qualifications, technical knowledge, project experience and areas of interest. These details can be compared with the requirements listed in internship advertisements to identify potential matches.

Profile matching can help students discover internships that correspond to their qualifications and interests. The review highlights the importance of selecting suitable profile information and evaluating recommendation quality. This research is relevant to TalentTrack because the application aims to help students compare their existing skills with the requirements of a selected job role. The comparison can guide preparation, although it cannot independently prove a student's practical competence. An important consideration is the quality of profile data. If a student's skills are incomplete or an internship description is unclear, the matching results may be less useful. Evaluation is also necessary to understand whether recommendations are relevant to students rather than simply similar in wording to their profiles.

This topic is relevant to TalentTrack because students preparing for placements often need to understand the skills expected for a selected role. Comparing a resume with a job description can help identify requirements that deserve more preparation. The system should provide useful guidance without assuming that a textual match necessarily represents practical competence.An important part of a job recommendation system is how it represents job seekers and job opportunities

2.9 Representing Jobs and Job Seekers in AI-Based Recommender Systems: A Literature Review

Author(s): Refer to the author list in the original ScienceDirect article
Year of Publication: 2026

This literature review examines how AI-based recommendation systems represent jobs, applicants and job postings. It compares knowledge-based, data-driven and hybrid approaches used for job matching. Knowledge-based systems use structured concepts and relationships, while data-driven systems learn patterns from available information. Hybrid approaches attempt to combine the strengths of both methods. An important part of a job recommendation system is how it represents job seekers and job opportunities. A candidate profile may contain skills, qualifications, projects and experience, while a job profile may contain required skills, responsibilities and eligibility criteria. These details need to be represented in a comparable form before a system can identify relationships between them.

The study identifies challenges involving inconsistent skill representations, data bias, limited interpretability and interoperability. It also highlights the importance of representing skills and competencies clearly when matching candidates with job requirements. This research relates to TalentTrack because accurate skill identification and comparison are necessary to provide meaningful resume-analysis results. Different approaches can be used to compare candidate and job information. Keyword matching focuses on shared terms, while semantic methods attempt to identify relationships between words and phrases with similar meanings. More complex approaches may combine different sources of information to improve matching. However, a system must be evaluated to ensure that its results are relevant and understandable.

This research area relates to TalentTrack's resume and job-description comparison process. The application aims to identify skills represented in the resume and requirements that may need further attention. Clear explanations of the comparison results can help students understand how their resume relates to the selected job role.

2.10 Trends and Characteristics of Career Recommendation Systems for Fresh Graduated Students

Author(s): Refer to the author list in the original IEEE Xplore publication
Year of Publication: 2022

This study reviews career recommendation systems designed for recent graduates. It examines the characteristics of these systems, the techniques used to recommend careers and the information considered when matching users with opportunities. The review reports that hybrid recommendation approaches and text-mining techniques are frequently used in the studies it examined. Fresh graduates may find it difficult to select suitable career paths because they have limited professional experience and may not be familiar with the requirements of different job roles. Career recommendation systems attempt to support this decision by considering information such as education, skills, interests and career preferences. These systems can help students explore possible opportunities in a more organized way.

The study also discusses the features commonly considered in career recommendation, including job profiles and course profiles, along with evaluation measures used to assess recommendations. Its findings are relevant to TalentTrack because final-year students need career guidance that considers their skills and target roles. TalentTrack aims to support this need by connecting resume analysis with preparation activities. Career guidance is more useful when recommendations are connected to practical preparation. Students may need to improve technical knowledge, practise aptitude questions and prepare for interviews after identifying a target role. A system that supports both career exploration and preparation can therefore address a broader need than a system that only lists job opportunities.

This theme is relevant to TalentTrack because the application aims to connect resume analysis and job requirements with placement preparation activities. Its intended purpose is to help students identify areas for improvement and organize their preparation. The effectiveness of the application must still be assessed through actual testing and user feedback.

CHAPTER 3

EXISTING SYSTEM

3.1 EXISTING SYSTEM

The existing placement preparation system mainly depends on traditional career guidance, manual resume checking, and separate online platforms for aptitude tests, coding practice, and interview preparation. Students generally use different websites and applications to improve their technical knowledge, prepare resumes, search for job opportunities, and practise interview questions. Since these activities are handled separately, students may find it difficult to organize their preparation and monitor their overall progress.

In many existing systems, resume evaluation is performed through basic keyword matching or manual review. These methods can identify certain skills and qualifications, but they may not fully understand the context of a student's experience or the relationship between the resume and a particular job description. Some placement portals focus primarily on managing recruitment activities and student records rather than providing personalized career preparation.



Figure 3.1 Existing System

3.2 DISADVANTAGES

- Preparation resources are spread across multiple platforms.
- Resume skills may not be systematically compared with job requirements.
- Students may have difficulty identifying missing or underdeveloped skills.
- Learning plans may be generic rather than related to a target role.
- Practice results and preparation history may be stored separately.
- Students may struggle to prioritize topics before recruitment assessments.
- Career guidance may depend heavily on informal advice or manual research.
- Manual resume evaluation and placement preparation require considerable time and effort.
- General career guidance may not match each student's individual skills and learning needs.
- Students may struggle to identify the skills missing for their target job role.

CHAPTER 4

PROBLEMS IDENTIFIED

In the current competitive job market, campus placement preparation has become an important challenge for college students. Students are expected to possess technical knowledge, programming skills, communication abilities, aptitude skills, and interview confidence to secure employment opportunities. However, many students find it difficult to identify the skills required for their target job roles and understand whether their current knowledge meets industry expectations. The absence of proper guidance and an organized preparation strategy can make the placement preparation process challenging.

One of the major problems is the lack of an integrated platform for placement preparation. Students often use different websites and applications for resume creation, job searching, aptitude practice, coding exercises, technical learning, and interview preparation. Managing these activities across multiple platforms can consume considerable time and make it difficult to maintain a consistent preparation schedule. Students may also find it challenging to monitor their progress and identify the areas in which they need further improvement.

Another important problem is the difficulty in evaluating resumes according to specific job requirements. A resume may contain educational qualifications, technical skills, certifications, projects, and internship experience, but students may not know whether these details match the expectations of a particular employer. Traditional resume checking methods may depend on manual evaluation or basic keyword matching. Such methods may not always identify the relationship between a student's existing capabilities and the complete requirements of a job description.

CHAPTER 5

PROPOSED SYSTEM

5.1 PROPOSED SYSTEM OVERVIEW

TalentTrack is proposed as an Android-based placement preparation and career guidance system. It combines account access, profile management, resume handling, job-description input, resume analysis, skill-gap identification, and preparation activities in a unified workflow.

The user can sign in, maintain relevant profile information, upload a resume, and provide a job description or select a supported job role. The analysis component compares the skills identified in the resume with the requirements mentioned in the job description. Where the analysis service returns valid results, the application can display matching skills, potential missing skills, and a readiness estimate.

The Readiness Engine is intended to help students organise preparation activities based on identified needs. The application may include practice tests, coding preparation, interview planning, and progress tracking according to the available implementation.

5.2 USER AUTHENTICATION AND PROFILE MANAGEMENT

The User Authentication and Profile Management module provides a secure and convenient way for students to access the TalentTrack application. It allows new users to register and existing users to log in using their credentials. The module is designed to provide a personalized experience by maintaining the user's account and profile information.

The authentication process allows users to access the application after successful login. Google Sign-In can also be used to simplify the login process, if enabled and configured in the application. If a user forgets their password, the password recovery option helps them reset it through the registered email address.

The profile management feature allows students to maintain their personal and educational information. The profile may include the student's name, email address, phone number, date of birth, degree, department, CGPA, graduation year, career interests, and skills.

5.3 RESUME MANAGEMENT AND JOB DESCRIPTION HANDLING

he Resume Management and Job Description Handling module enables students to provide the information required for resume evaluation and job-specific preparation. The module allows users to select and upload their resume from the device using the file picker. The uploaded resume acts as the primary source of information about the student's education, technical skills, projects, certifications, and experience.

In addition to resume management, the module handles the job description associated with the student's target job role. A job description contains details about the responsibilities, qualifications, technical skills, and competencies expected by an employer. Providing a relevant job description is important because the same resume may match one job role but may not fully satisfy the requirements of another.

5.4 RESUME ANALYSIS AND SKILL GAP IDENTIFICATION

The Resume Analysis and Skill Gap Identification module is one of the key modules of TalentTrack. Its purpose is to help students understand how well their current qualifications and skills match the requirements of a selected job role. The analysis uses the uploaded resume and the corresponding job description as inputs.

During the analysis process, the student's resume information is examined to identify skills relevant to the selected job. These skills are compared with the requirements mentioned in the job description. Based on the comparison, the system aims to categorize the identified skills into existing skills and missing skills. Existing skills represent the competencies found in the resume that match the job requirements, while missing skills represent relevant requirements that are not identified in the submitted resume.

The module also aims to provide a readiness percentage to help students understand their level of alignment with the selected job description. This percentage should be displayed only after the analysis has completed successfully and should be based on the application's implemented scoring method.

5.5 PROPOSED SYSTEM ARCHITECTURE

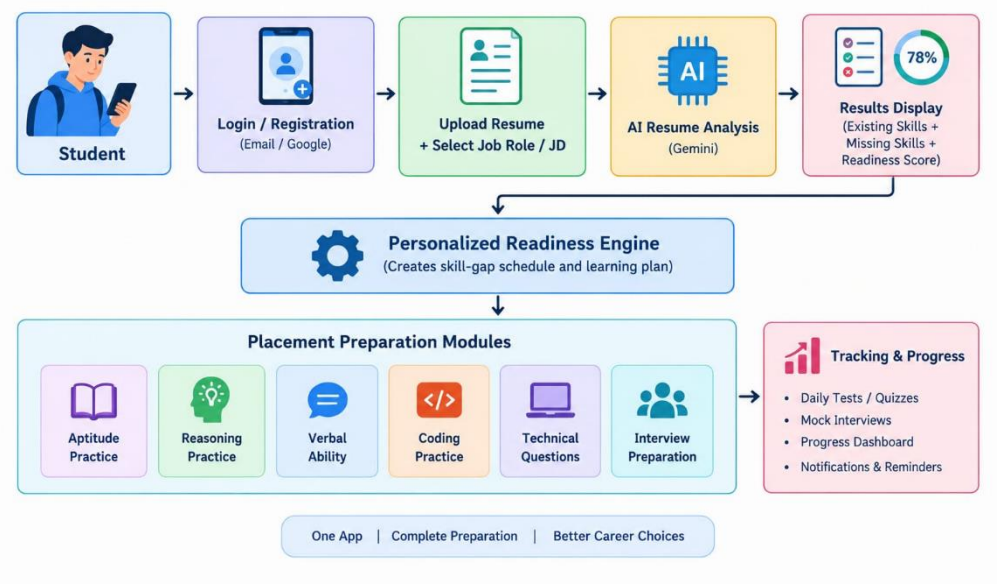


Figure 5.5 Proposed System

The presentation layer displays the application screens. The application layer coordinates screen state and user actions. Firebase Authentication manages supported sign-in operations. The AI analysis service processes resume and job-description text if integrated. The data layer stores information through the Firebase services configured in the application.

5.6 ADVANTAGES

- Provides a unified mobile platform for placement preparation.
- Connects resume information with job-role requirements.
- Helps identify potential skill gaps.
- Supports role-oriented preparation activities.
- Organises supported practice and interview-preparation modules.
- Provides a structured workflow for placement preparation.
- Can be extended with additional job roles and learning resources.

CHAPTER 6

SYSTEM REQUIREMENTS

6.1 HARDWARE REQUIREMENTS

• **Processor**

A computer with an Intel Core i5 processor or higher, or an equivalent AMD processor, is recommended for developing the TalentTrack application. The processor handles Android Studio, Gradle builds, Kotlin compilation, and application testing. A capable processor helps reduce build time and supports smooth development.

• **Memory (RAM)**

A minimum of 8 GB RAM is recommended, while 16 GB is preferred for smoother performance. Android Studio, Gradle, and the Android emulator require sufficient memory. Adequate RAM helps developers run the application, debug code, and test multiple screens efficiently.

• **Storage**

At least 256 GB of storage, preferably an SSD, is recommended. Storage is required for Android Studio, Android SDK components, emulator images, project source files, Gradle dependencies, and build outputs. SSD storage improves application startup and build performance.

• **Android Device**

An Android smartphone or emulator running a compatible Android version is required to install and test TalentTrack. A physical device helps verify login, profile management, resume file selection, job description entry, and the overall user interface.

• **Internet Connectivity**

A stable internet connection is required for downloading Android libraries and Gradle dependencies. It is also necessary for accessing Firebase services and any external AI analysis service configured in the application. Internet connectivity supports authentication, cloud data access, and resume analysis where these features are implemented.

- **Second Test Device**

A second Android device or emulator is recommended for testing login and account switching. Testing with different user accounts helps verify that each user's profile and application data are handled correctly. It is also useful for checking Google Sign-In across devices and identifying authentication configuration issues.

6.2 SOFTWARE REQUIREMENTS

- **Operating System**

The development environment can run on Windows, Linux, or macOS because Android Studio supports these operating systems. The TalentTrack application is designed to run on compatible Android devices.

- **Frontend (Kotlin and Jetpack Compose)**

Kotlin is used as the programming language, while Jetpack Compose is used to develop the application's user interface. Compose supports the creation of screens such as splash, login, registration, home dashboard, profile, resume management, resume analysis, and placement preparation. Its composable functions help create reusable and maintainable UI components.

- **Backend (Firebase)**

Firebase provides backend services for application authentication and cloud-based data management, depending on the services configured in the project. Firebase Authentication supports user registration, login, and Google Sign-In. Firebase services can also be used to store user profiles and other application data. The exact services used should match the implementation in the TalentTrack project.

- **Architecture (MVVM, ViewModel, StateFlow and Coroutines)**

TalentTrack follows the Model–View–ViewModel (MVVM) architecture to separate the user interface from application logic and data operations. ViewModels manage screen-related state and business logic. StateFlow can expose changing state to Jetpack Compose, while Kotlin Coroutines support asynchronous operations such as authentication, database requests, and AI service calls.

- **Libraries (Navigation Compose and Firebase SDK)**

Navigation Compose is used to navigate between screens such as login, home, profile, resume analysis, and practice modules. Firebase SDKs provide access to the

Firebase services configured in the application. Additional libraries may be used for PDF or document selection, resume processing, and network communication according to the actual implementation.

- **Database and Storage**

Firebase services can store and manage user profiles, resume details, readiness results, and preparation progress when configured in the application. Firebase Storage may be used for uploaded files if it is part of the implemented system. Appropriate security rules and authentication checks are required to protect user information and uploaded resumes.

- **Development Environment**

Android Studio is used for writing Kotlin code, designing Jetpack Compose screens, debugging, running emulator tests, and building the application. Gradle manages the project's build process and dependencies. Firebase Console is used to configure authentication and the Firebase services used by TalentTrack. Git and GitHub can be used for version control and project backup.

CHAPTER 7

SYSTEM IMPLEMENTATIONS

7.1 LIST OF MODULES

- User Authentication and Account Management
- Home Dashboard and Navigation
- Profile, Education, and Skills Management
- Resume Management
- Job Description Selection and Handling
- Resume Analysis and Skill Gap Engine
- Readiness Estimate and Preparation Planning
- Practice Tests and Coding Preparation

7.2 MODULE DESCRIPTION

7.2.1 User Authentication and Account Management Module

The User Authentication and Account Management module allows students to register and log in to the TalentTrack application. It provides access to the application through user credentials and supports Google Sign-In if configured. The module also includes account-related functions such as password recovery and logout, where implemented. It helps provide a personalized and organized application experience for students preparing for placements.

7.2.2 Home Dashboard and Navigation

The Home Dashboard and Navigation module serves as the main interface of the TalentTrack application. It provides access to the major features, including profile management, resume management, resume analysis, skill gap identification, and placement preparation. The dashboard helps students navigate between the available modules easily. Its organized layout allows users to access the required features without searching through multiple screens.

7.2.3 Profile, Education, And Skills Management

The Profile, Education, and Skills Management module allows students to maintain their personal, academic, and skill-related information. The profile can include details such as name, email address, phone number, date of birth, degree, department, CGPA, graduation year, and career interests. Students can also maintain their programming languages, technical skills, and soft skills. This information helps organize their qualifications and supports resume-based placement preparation.

7.2.4 Resume Management

The Resume Management module enables students to upload and manage their resumes through the application. The resume contains important information such as educational qualifications, technical skills, projects, certifications, and internship experience. Students can select their resume from the device using the file picker. The uploaded resume acts as the primary input for the resume analysis process and helps the application identify skills relevant to the student's target job role.

7.2.5 Job Description Handling

The Job Description Selection and Handling module manages the job requirements used during resume evaluation. Students can select a target company and job role and provide the corresponding job description, depending on the options available in the application. A job description contains information about required qualifications, technical skills, responsibilities, and competencies. This information serves as a reference for comparing the student's resume with the expectations of the selected job role.

7.2.6 Resume Analysis and Skill Gap Engine

The Job Description Selection and Handling module manages the job requirements used during resume evaluation. Students can select a target company and job role and provide the corresponding job description, depending on the options available in the application. A job description contains information about required

qualifications, technical skills, responsibilities, and competencies. This information serves as a reference for comparing the student's resume with the expectations of the selected job role.

7.2.7 Readiness and Preparation Planning

The Readiness Estimate and Preparation Planning module aims to help students understand their preparation level for a selected job role. After successful resume analysis, the application can display a readiness percentage based on its implemented scoring method. Using the identified skill gaps, the system can organize relevant preparation activities into a structured plan. This may include technical learning, coding practice, aptitude preparation, and interview preparation. The module is intended to help students focus on areas requiring improvement and follow a more organized preparation process.

7.2.8 Practice and Coding Preparation

The Practice Tests and Coding Preparation module supports students in improving their knowledge and problem-solving abilities. It may include coding exercises, aptitude questions, verbal ability, logical reasoning, and technical multiple-choice questions, depending on the features implemented in the application. Students can use these activities to practise important placement topics and strengthen their understanding. Regular practice can help students become more familiar with placement assessment formats and identify topics that require additional preparation.

CHAPTER 8
SYSTEM TESTING

8.1 UNIT TESTING

Unit testing checks individual functions or components separately. In TalentTrack, this may include required-field validation, file validation, navigation logic, and skill comparison functions. Each unit should be tested with valid and invalid inputs to verify that it behaves as expected.

8.2 INTEGRATION TESTING

Integration testing verifies that the application's components work together. Examples include checking authentication with Firebase, providing resume and job-description data to the analysis service, and displaying valid analysis results in the interface.

8.3 SYSTEM TESTING

System testing evaluates the complete application from the user's point of view. The workflow can be tested from login through profile access, resume selection, job-description input, analysis, and supported preparation activities.

8.4 PERFORMANCE TESTING

Performance testing evaluates whether the application remains responsive during normal use. It can include opening the dashboard, navigating between screens, selecting a resume, and waiting for analysis results.

8.5 USABILITY TESTING

Usability testing evaluates whether students can understand and use the interface. The labels, buttons, navigation, input instructions, and error messages should be clear. Feedback from actual users can be used to improve the interface and simplify the preparation workflow.

CHAPTER 9
RESULT AND DISCUSSION

TalentTrack is designed to combine several placement preparation activities within a single Android application. Its main workflow connects account access, profile information, resume handling, job-description input, skill analysis, and preparation activities. This structure aims to reduce the need to manage every preparation activity separately.

Resume and job-description comparison can help students identify the skills represented in their resumes and the requirements that may need further attention. These findings are useful for planning preparation, but they should not be interpreted as proof of practical ability. The usefulness of the results depends on the quality of the source documents and the analysis service.

The application was also made available through Google Play Console testing. Testing on different devices is important because authentication and other integrations may behave differently depending on signing configuration, account settings, and network conditions.

The integrated approach of TalentTrack aims to make placement preparation more organized by connecting resume analysis with skill-gap identification and preparation activities. However, the results should be considered guidance for preparation rather than a guarantee of placement success.

The accuracy of resume and job-description analysis depends on the quality and format of the uploaded documents and the analysis service. The identified skill gaps may not fully represent a student's practical knowledge or problem-solving ability. In addition, authentication and other integrated services may require configuration and testing across different devices.

The following test cases can be used to evaluate the main functions of TalentTrack. The actual result column should be completed based on the tests performed on the application.

Test Case	Expected Result	Actual Result
User Login	The user should be able to access the application after successful authentication.	Enter the tested result
Google Sign-In	The selected Google account should be authenticated successfully.	Enter the tested result
Profile Management	The user should be able to view or update profile information.	Enter the tested result
Resume Upload	The selected resume file should be accepted by the application.	Enter the tested result
Job Description Input	The job description should be available for analysis.	Enter the tested result
Resume Analysis	Analysis results should be displayed after processing.	Enter the tested result
Skill Gap Analysis	Identified skills and skills requiring attention should be displayed.	Enter the tested result
Practice Module	The selected practice section should open correctly.	Enter the tested result

TalentTrack brings important placement preparation activities together in one Android application. Resume and job-description comparison can help students identify areas for improvement and plan their preparation. Further testing and refinement can improve reliability, usability, and the consistency of results.

CHAPTER 10
CONCLUSION AND FUTURE ENHANCEMENTS

10.1 CONCLUSION

TalentTrack is an Android-based placement preparation and career guidance system designed to help students organise their preparation for recruitment. It connects profile management, resume handling, job-description comparison, skill-gap identification, and placement practice within a single application workflow.

The system aims to help students understand the skills mentioned in their resumes in relation to the requirements of a target job. Potential skill gaps can be used to prioritise learning and practice activities. The use of Kotlin, Jetpack Compose, MVVM, Firebase, and an AI analysis service where configured demonstrates how mobile application development and cloud-based services can support an educational use case.

TalentTrack is intended to provide preparation guidance rather than make hiring decisions. Its output depends on the quality of the input documents and the reliability of the configured analysis process. Further testing and improvement can increase the reliability and usefulness of the application.

10.2 FUTURE ENHANCEMENTS

In the future, TalentTrack can be improved by supporting additional resume formats and enhancing the extraction of skills, educational qualifications, and project details. The job role database can be expanded with verified job descriptions from more companies and entry-level positions. The skill-matching process can be enhanced to recognise synonyms and related skills in resumes and job descriptions. Personalized preparation plans can be improved by considering students' practice results and learning progress. A larger question bank containing coding, aptitude, reasoning, verbal ability, and technical questions with explanations can be added to support placement preparation. The mock interview module can provide structured questions and useful feedback based on the implemented evaluation method. Progress analytics can help students understand their preparation status and identify areas that require improvement. Error handling can be strengthened to manage network failures, authentication problems, and AI service errors effectively. Furthermore, cloud-based synchronization, smart notifications, multilingual support, advanced career recommendations, and offline practice facilities can be introduced to improve accessibility and usability. Additional privacy controls, secure data handling, and clear account-deletion procedures can also be implemented to protect user information. These enhancements can make TalentTrack a more reliable, accessible, and effective placement preparation and career guidance application.

APPENDIX – A
SOURCE CODE

LoginScreen.kt

```
package com.mad.talenttrack.ui.screens
import android.widget.Toast
import androidx.activity.compose.rememberLauncherForActivityResult
import androidx.activity.result.contract.ActivityResultContracts
import androidx.compose.foundation.layout.*
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.res.stringResource
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.input.PasswordVisualTransformation
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.lifecycle.viewmodel.compose.viewModel
import com.google.android.gms.auth.api.signin.GoogleSignIn
import com.google.android.gms.auth.api.signin.GoogleSignInOptions
import com.google.android.gms.common.api.ApiException
import com.mad.talenttrack.R
import com.mad.talenttrack.viewmodel.AuthViewModel

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun LoginScreen(
    onLoginSuccess: () -> Unit,
    onRegisterClick: () -> Unit,
```

```
    viewModel: AuthViewModel = viewModel()
) {
    var email by remember { mutableStateOf("") }
    var password by remember { mutableStateOf("") }
    var showForgotDialog by remember { mutableStateOf(false) }
    var forgotEmail by remember { mutableStateOf("") }

    val loading by viewModel.loading
    val errorMessage by viewModel.errorMessage
    val successMessage by viewModel.successMessage

    val context = LocalContext.current

    val webClientId = stringResource(R.string.default_web_client_id)
    val gso = remember(webClientId) {
        GoogleSignInOptions.Builder(
            GoogleSignInOptions.DEFAULT_SIGN_IN
        )
            .requestEmail()
            .requestProfile()
            .requestIdToken(webClientId)
            .build()
    }
    val googleSignInClient = remember(gso) {
        GoogleSignIn.getClient(context, gso)
    }

    val googleSignInLauncher =
        rememberLauncherForActivityResult(
            contract = ActivityResultContracts.StartActivityForResult()
        ) { result ->

            val intentData = result.data
```

```
    val task =
        GoogleSignIn.getSignedInAccountFromIntent(intentData)

    try {
        val account =
            task.getResult(ApiException::class.java)

        val idToken = account.idToken

        if (!idToken.isNullOrEmpty()) {
            viewModel.signInWithGoogle(
                idToken,
                onLoginSuccess
            )
    } catch (e: ApiException) {

        val errorMsg = when (e.statusCode) {
            10 ->
                "Developer Error 10: Check Firebase SHA-1 and Web Client ID."
        if (e.statusCode != 12501) {
            Toast.makeText(
                context,
                errorMsg,
                Toast.LENGTH_LONG
            ).show()
        }
    }
}
    Spacer(Modifier.height(16.dp))
    OutlinedButton(
        onClick = {
            viewModel.clearMessages()
```

```
                googleSignInClient.signOut()
                    .addOnCompleteListener {

                        // Open Google account chooser
                        googleSignInLauncher.launch(
                            googleSignInClient.signInIntent
                        )
                    }
            },
            modifier = Modifier.fillMaxWidth()
        ) {
            Text("Continue with Google")
@Preview(showBackground = true)
@Composable
fun LoginScreenPreview() {
    LoginScreen(
        onLoginSuccess = {},
        onRegisterClick = {}
    )
}
```

HomeScreen.kt

```
package com.mad.talenttrack.ui.screens
import androidx.compose.foundation.Canvas
import androidx.compose.foundation.background
import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.shape.CircleShape
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.verticalScroll
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.automirrored.filled.Article
import androidx.compose.material.icons.automirrored.filled.MenuBook
```

```
import androidx.compose.material.icons.filled.*
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.Brush
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.StrokeCap
import androidx.compose.ui.graphics.drawscope.Stroke
import androidx.compose.ui.graphics.vector.ImageVector
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.style.TextAlign
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.lifecycle.viewmodel.compose.viewModel
import com.google.firebase.auth.FirebaseAuth
import com.mad.talenttrack.data.model.LearningTask
import com.mad.talenttrack.ui.theme.*
import com.mad.talenttrack.viewmodel.ProfileViewModel
import java.text.SimpleDateFormat
import java.util.*

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun HomeScreen(
    onMenuClick: () -> Unit,
    onLogout: () -> Unit,
    onActionClick: (String) -> Unit,
    profileViewModel: ProfileViewModel = viewModel()
) {
    val userProfile by profileViewModel.userProfile
    val user = FirebaseAuth.getInstance().currentUser
```

```
val name =
    userProfile?.name
        ?: user?.email?.split("@")?.firstOrNull()
        ?: "Student"

val currentDay = remember {
    SimpleDateFormat(
        "EEEE",
        Locale.getDefault()
    ).format(Date())
}

val todaysTasks =
    userProfile?.learningSchedule?.get(currentDay)
        ?: emptyList()

Scaffold(
    topBar = {
        CenterAlignedTopAppBar(

            title = {
                Text(
                    "TalentTrack",
                    fontWeight = FontWeight.Bold,
                    color = AccentBlue,
                    fontSize = 24.sp
                )
            },

            navigationIcon = {
                IconButton(
                    onClick = onMenuClick
                ) {
                    Icon(
```

```
                    Icons.Default.Menu,
                    contentDescription = "Menu",
                    tint = Color.White
                )
            }
        },
        Spacer(
            modifier = Modifier.height(16.dp)
        )
        QuickActionsSection(
            onActionClick
        )

        Spacer(
            modifier = Modifier.height(16.dp)
        )          BottomQuoteCard()

        Spacer(
            modifier = Modifier.height(32.dp)
        )
    }
  }
}
@Composable
fun HeaderSection(
    name: String
) {

    Row(

        modifier = Modifier
            .fillMaxWidth()
            .padding(vertical = 8.dp),
```

```
    horizontalArrangement =
        Arrangement.SpaceBetween,

    verticalAlignment =
        Alignment.CenterVertically

) {

    Column {

        Text(
            text = "Hello, $name 👏",
            style =
                MaterialTheme.typography.headlineMedium,
            fontWeight =
                FontWeight.Bold,
            color = Color.White
        )

        Text(
            text =
                "\"Believe you can and you're halfway there.\"",
            style =
                MaterialTheme.typography.bodySmall,
            color = TextSecondary
        )
    }

    Box(

        modifier = Modifier
            .size(100.dp)
            .background(
                Brush.radialGradient(
```

```
                    colors =
                        listOf(
                            AccentBlue.copy(alpha = 0.2f),
                            Color.Transparent
                        )
                )
            ),

        contentAlignment =
            Alignment.Center

    ) {

        Icon(
            Icons.Default.Laptop,
            contentDescription = null,
            tint =
                AccentBlue.copy(alpha = 0.6f),
            modifier =
                Modifier.size(60.dp)
        )
    }
  }
}

@Composable
fun ReadinessCard(
  score: Int,
  isAnalyzed: Boolean,
  onClick: () -> Unit
) {
  val hasPreparationStarted =
    isAnalyzed && score > 0
```

```
val progress =
    if (hasPreparationStarted) {
        score / 100f
    } else {
        0f
    }

val statusText =
    when {

        !isAnalyzed ->
            "Not Analyzed"

        !hasPreparationStarted ->
            "Not Assessed"
    modifier =
        Modifier
            .fillMaxWidth()
            .clickable(
                onClick = onClick
            ),
            Text(
                if (isAnalyzed) {
                    "View Details >"
                } else {
                    "Start Analysis >"
                },                Modifier.height(20.dp)
        )
                Canvas(
                    modifier =
                        Modifier.fillMaxSize()
                ) {
                    drawArc(
                        color =
```

```
            Color.DarkGray
                .copy(alpha = 0.3f),
        startAngle = 0f,
        sweepAngle = 360f,
        useCenter = false,
        style =
            Stroke(
                width =
                    10.dp.toPx(),

                cap =
                    StrokeCap.Round
            )
    )
    if (hasPreparationStarted) {
        AccentGreen,
            startAngle =
                -90f,
            sweepAngle =
                progress * 360f,
            useCenter = false,
            style =
                Stroke(
                    width =
                        10.dp.toPx(),
                    cap =
                        StrokeCap.Round
                )
        )
    }
}

Column(
```

```
            horizontalAlignment =
                Alignment.CenterHorizontally

        ) {
            Text(
                when {
                    !isAnalyzed ->
                        "--"
                    !hasPreparationStarted ->
                        "--"
                    else ->
                        "$score%"
                },
                color =
                    Color.White,
                fontWeight =
                    FontWeight.Bold,
                fontSize =
                    24.sp
            )
            Text(
                if (hasPreparationStarted) {
                    "Ready"
                } else {
                    "Readiness"
                },
                color =
                    TextSecondary,
                fontSize =
                    12.sp
            )
        }
    }
```

```
Spacer(
    modifier =
        Modifier.width(24.dp)
)
Column {
    Text(
        statusText,
        color =
            Color.White,
        fontWeight =
            FontWeight.Bold
    )
    Spacer(
        modifier =
            Modifier.height(4.dp)
    )
    Text(
        motivationText,
        color =
            TextSecondary,
        fontSize =
            12.sp,
        lineHeight =
            18.sp
    )
    if (hasPreparationStarted) {
        Spacer(
            modifier =
                Modifier.height(16.dp)
        )
        LinearProgressIndicator(
            progress = {
                progress
            },
```

```
                modifier =
                    Modifier
                        .fillMaxWidth()
                        .height(8.dp)
                        .clip(CircleShape),

                color =
                    AccentGreen,

                trackColor =
                    Color.DarkGray
                        .copy(alpha = 0.3f)
            )

            Text(
                "Target: 100%",
                color =
                    TextSecondary,
                fontSize =
                    11.sp,
                modifier =
                    Modifier.padding(
                        top = 6.dp
                    )
            )
        }
    }
  }
}
@Composable
fun TasksCard(
```

```
    tasks: List<LearningTask>,
    modifier: Modifier =
        Modifier,
    onViewAll: () -> Unit,
    onTaskToggle: (String) -> Unit
) {
    Card(
        modifier =
            modifier,
        shape =
            RoundedCornerShape(20.dp),
        colors =
            CardDefaults.cardColors(
                containerColor =
                    CardBackground
            )
    ) {
        Column(
            modifier =
                Modifier.padding(16.dp)
        ) {
            Text(
                "Today's Tasks",
                color =
                    Color.White,
                fontWeight =
                    FontWeight.Bold,
                fontSize =
                    16.sp
            )
            Spacer(
                modifier =
                    Modifier.height(16.dp)
```

```
)
if (tasks.isEmpty()) {
    Text(
        "No tasks for today.",
        color =
            TextSecondary,
        fontSize =
            12.sp
    )
} else {
    tasks
        .take(3)
        .forEach { task ->
            val icon =
                when (task.type) {
                    "Learning" ->
                        Icons.Default.Psychology
                    "Practice" ->
                        Icons.Default.Code
                    "Coding" ->
                        Icons.Default.Code
                    "Interview" ->
                        Icons.Default.QuestionAnswer
                    "Assessment" ->
                        Icons.Default.Assessment
                    "Mock Interview" ->
                        Icons.Default.Mic
                    else ->
                        Icons.Default.Description
                }

            val color =
                when (task.type) {
```

```
                "Learning" ->
                    AccentPurple
                "Practice" ->
                    AccentGreen
                "Coding" ->
                    AccentGreen
                "Interview" ->
                    AccentOrange
                "Assessment" ->
                    AccentBlue
                "Mock Interview" ->
                    AccentBlue
                else ->
                    AccentOrange
            }
        TaskItemRow(
            icon =
                icon,
            title =
                task.title,
            color =
                color,
            isChecked =
                task.isCompleted,
            onToggle = {
                onTaskToggle(
                    task.id
                )
            }
        )
    }
}

Spacer(
```

```
                modifier =
                    Modifier.height(12.dp)
            )
            Text(
                "View All Tasks >",
                color =
                    AccentBlue,
                fontSize =
                    12.sp,
                modifier =
                    Modifier
                        .align(Alignment.End)
                        .clickable(
                            onClick = onViewAll
                        )
            )
        }
    }
}
@Composable
fun TaskItemRow(
    icon: ImageVector,
    title: String,
    color: Color,
    isChecked: Boolean,
    onToggle: () -> Unit
) {
    Row(
        modifier =
            Modifier
                .fillMaxWidth()
                .padding(vertical = 6.dp),
        verticalAlignment =
            Alignment.CenterVertically
```

```
) {      contentAlignment =
            Alignment.Center
    ) {
        Icon(
            icon,
            contentDescription =
                null,
            tint =
                color,
            modifier =
                Modifier.size(18.dp)
        )
    }
    Spacer(
        modifier =
            Modifier.width(12.dp)
    )
    Text(
        title,
        color =
            Color.White,
        fontSize =
            13.sp,
        modifier =
            Modifier.weight(1f)
    )
    Checkbox(
        checked =
            isChecked,
        onCheckedChange = {
            onToggle()
        },
```

```
            colors =
                CheckboxDefaults.colors(
                    uncheckedColor =
                        Color.Gray
                )
        )
    }
}
@Composable
fun QuickActionsSection(
    onActionClick: (String) -> Unit
) {
    Column {
        Text(
            "Quick Actions",
            color =
                Color.White,
            fontWeight =
                FontWeight.Bold,
            fontSize =
                16.sp
        )
        Card(
            modifier =
                Modifier.fillMaxWidth(),
            shape =
                RoundedCornerShape(24.dp),
            colors =
                CardDefaults.cardColors(
                    containerColor =
                        CardBackground
                )
        ) {
```

```
Row(
    modifier=Modifier
        .fillMaxWidth()
        .padding(20.dp),
    horizontalArrangement =
        Arrangement.SpaceBetween
) {
    ActionItem(
        Icons.AutoMirrored.Filled.MenuBook,
        "Practice",
        AccentPurple
    ) {
        onActionClick("practice")
    }
    ActionItem(
        Icons.Default.BarChart,
        "Daily Test",
        AccentBlue
    ) {
        onActionClick("dailyTest")
    }
    ActionItem(
        Icons.Default.EventAvailable,
        "Interview\nPlanner",
        AccentOrange
    ) {
        onActionClick("planner")
    }

    ActionItem(
        Icons.AutoMirrored.Filled.Article,
        "Resume",
        AccentGreen
    ) {
```

```
                onActionClick("resume")
            }
            ActionItem(
                Icons.Default.Mic,
                "Mock\nInterview",
                AccentBlue
            ) {
                onActionClick("mockInterview")
            }
        }
    }
}
@Composable
fun ActionItem(
    icon: ImageVector,
    title: String,
    color: Color,
    onClick: () -> Unit
) {
    Column(
        horizontalAlignment =
            Alignment.CenterHorizontally,
        modifier =
            Modifier
                .width(50.dp)
                .clickable(
                    onClick = onClick
                )
    ) {
        Box(
            modifier =
                Modifier
                    .size(45.dp)
```

```
                .background(
                    color.copy(alpha = 0.1f),
                    RoundedCornerShape(12.dp)
                ),
            contentAlignment =
                Alignment.Center
        ) {
            Icon(
                icon,
                contentDescription =
                    null,
                tint =
                    color,
                modifier =
                    Modifier.size(24.dp)
            )
        }
        Spacer(
            modifier =
                Modifier.height(8.dp)
        )
        Text(
            title,
            color =
                TextSecondary,
            fontSize =
                9.sp,
            textAlign =
                TextAlign.Center,
            lineHeight =
                12.sp
        )
    }
}
```

```
@Composable
        fun BottomQuoteCard() {
            Card(
                modifier =
                    Modifier.fillMaxWidth(),
                shape =
                    RoundedCornerShape(20.dp),
                colors =
                    CardDefaults.cardColors(
                        containerColor =
                            AccentBlue.copy(alpha = 0.05f)
                    )
            ) {
                Column(
                    modifier =
                        Modifier.padding(20.dp)
                ) {
                    Icon(
                        Icons.Default.FormatQuote,
                        contentDescription =
                            null,
                        tint =
                            AccentBlue,
                        modifier =
                            Modifier.size(28.dp)
                    )
                    Text(
                        "The best way to predict your future is to create it.",
                        color =
                            Color.White,
                        fontSize =
                            16.sp,
                        lineHeight =
                            24.sp,
```

```
                modifier =
                    Modifier.padding(
                        top = 8.dp
                    )
            )
            Text(
                "- Abraham Lincoln",
                color =
                    TextSecondary,
                fontSize =
                    12.sp,
                modifier =
                    Modifier
                        .align(Alignment.End)
                        .padding(top = 12.dp)
            )
        }
    }
}
```

MockInterviewScreen.kt

```
Row(modifier=Modifier.fillMaxWidth(),horizontalArrangement=

Arrangement.spacedBy(12.dp)) {
        Card(
            modifier = Modifier.weight(1f),
            shape = RoundedCornerShape(16.dp),
            colors = CardDefaults.cardColors(containerColor = CardBackground)
        ) {
            Column(modifier = Modifier.padding(16.dp)) {
                Row(verticalAlignment = Alignment.CenterVertically) {
                    Icon(Icons.Default.ThumbUp, contentDescription = null, tint = AccentGreen,
modifier = Modifier.size(16.dp))
                    Spacer(Modifier.width(6.dp))
                    Text("Strong Areas", color = AccentGreen, fontWeight = FontWeight.Bold,
fontSize = 12.sp)
                }
                Spacer(Modifier.height(8.dp))
```

```
                summary.strongAreas.forEach { area ->
                    Text("• $area", color = Color.White, fontSize = 11.sp, lineHeight = 15.sp)
                }
            }
        }

        Card(
            modifier = Modifier.weight(1f),
            shape = RoundedCornerShape(16.dp),
            colors = CardDefaults.cardColors(containerColor = CardBackground)
        ) {
            Column(modifier = Modifier.padding(16.dp)) {
                Row(verticalAlignment = Alignment.CenterVertically) {
                    Icon(Icons.Default.Build, contentDescription = null, tint = AccentOrange,
modifier = Modifier.size(16.dp))
                    Spacer(Modifier.width(6.dp))
                    Text("Needs Focus", color = AccentOrange, fontWeight = FontWeight.Bold,
fontSize = 12.sp)
                }
                Spacer(Modifier.height(8.dp))
                summary.needsImprovement.forEach { area ->
                    Text("• $area", color = Color.White, fontSize = 11.sp, lineHeight = 15.sp)
                }
            }
        }
    }
    if (summary.codingAreasToImprove.isNotEmpty()) {
        Card(
            modifier = Modifier.fillMaxWidth(),
            shape = RoundedCornerShape(16.dp),
            colors = CardDefaults.cardColors(containerColor = CardBackground)
        ) {
            Column(modifier = Modifier.padding(16.dp)) {
                Row(verticalAlignment = Alignment.CenterVertically) {
                    Icon(Icons.Default.Code, contentDescription = null, tint = AccentOrange,
modifier = Modifier.size(16.dp))
                    Spacer(Modifier.width(6.dp))
                    Text("Coding Areas To Improve:", color = Color.White, fontWeight =
FontWeight.SemiBold, fontSize = 13.sp)
                }
                Spacer(Modifier.height(8.dp))
                summary.codingAreasToImprove.forEach { item ->
                    Text("• $item", color = TextSecondary, fontSize = 11.sp, lineHeight = 15.sp)
                }
            }
        }
    }
    if (summary.topSkillsToRevise.isNotEmpty()) {
        Card(
            modifier = Modifier.fillMaxWidth(),
```

```
            shape = RoundedCornerShape(16.dp),
            colors = CardDefaults.cardColors(containerColor = CardBackground)
        ) {
            Column(modifier = Modifier.padding(16.dp)) {
                Text("Top Skills To Revise:", color = Color.White, fontWeight =
FontWeight.SemiBold, fontSize = 13.sp)
                Spacer(Modifier.height(8.dp))
                Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
                    summary.topSkillsToRevise.forEach { skill ->
                        Surface(
                            shape = RoundedCornerShape(20.dp),
                            color = AccentBlue.copy(alpha = 0.2f),
                            border = androidx.compose.foundation.BorderStroke(1.dp, AccentBlue)
                        ) {
                            Text(
                                text = skill,
                                color = Color.White,
                                fontSize = 11.sp,
                                modifier = Modifier.padding(horizontal = 10.dp, vertical = 4.dp)
                            )
                        }
                    }
                }
            }
        }
    }
    Card(
        modifier = Modifier.fillMaxWidth(),
        shape = RoundedCornerShape(16.dp),
        colors = CardDefaults.cardColors(containerColor = CardBackground)
    ) {
        Column(modifier = Modifier.padding(16.dp)) {
            Text("Recruiter Final Feedback", color = AccentBlue, fontWeight =
FontWeight.Bold, fontSize = 13.sp)
            Spacer(Modifier.height(6.dp))
            Text(summary.finalFeedback, color = Color.White, fontSize = 12.sp, lineHeight =
18.sp)
        }
    }
    Row(
        modifier = Modifier.fillMaxWidth(),
        horizontalArrangement = Arrangement.spacedBy(12.dp)
    ) {
        OutlinedButton(
            onClick = onRetake,
            modifier = Modifier.weight(1f).height(48.dp),
            shape = RoundedCornerShape(12.dp)
        ) {
            Text("Retake Interview")
        }
```

```
            Button(
                onClick = onDashboard,
                modifier = Modifier.weight(1f).height(48.dp),
                shape = RoundedCornerShape(12.dp),
                colors = ButtonDefaults.buttonColors(containerColor = AccentBlue)
            ) {
                Text("Return to Dashboard")
            }
        }
    }
}
```

AIService.kt

```
object AIService {
    private const val DIRECT_API_KEY = "AQ.Ab8RN6I-
GbTt47AqOGVMbV3Acx64NG8rfcyjjCWGkPNpl8oUDg"

    private val API_KEY: String
        get() = DIRECT_API_KEY.takeIf { it.isNotBlank() } ?:
BuildConfig.GEMINI_API_KEY

    private val client: Client by lazy {
        Client.builder()
            .apiKey(API_KEY)
            .build()
    }
    suspend fun analyzeResume(
        resumeText: String,
        jobDetails: String
    ): ResumeAnalysisResult? {

        android.util.Log.d(
            "TalentTrack_AI",
            "Analyze Resume started"
        )

        if (
            API_KEY.isBlank() ||
            API_KEY.contains("ADD_YOUR_KEY") ||
            API_KEY.contains("YOUR_KEY") ||
            API_KEY.length < 10
        ) {
            android.util.Log.w(
                "TalentTrack_AI",
                "Gemini API key missing or invalid, using LocalAnalyzer"
            )
```

```
    return LocalAnalyzer.analyze(resumeText, jobDetails)
}

val prompt = """
    You are a Professional Career Coach and Technical Recruiter.

    Analyze the following Resume against the Job Description.

    Job Details:
    $jobDetails

    Candidate Resume:
    $resumeText

    Provide:
    1. matchedSkills
    2. missingSkills
    3. matchPercentage (0-100)
    4. strengths
    5. weaknesses
    6. recommendedSkills
    7. skillPriorities (High, Medium, Low)
    8. summary

    IMPORTANT:
    - Do NOT calculate preparation readiness.
    - readinessScore must always be 0 because preparation
      cannot be measured from a resume alone.
    - Match percentage must be based only on resume skills
      compared with the job requirements.
    - Do not invent skills that are not present in the resume.
    - Return ONLY valid JSON.
    - Do not use markdown code fences.

    Return exactly this structure:

    {
      "matchedSkills": ["Java", "SQL"],
      "missingSkills": ["Spring Boot"],
      "matchPercentage": 75,
      "readinessScore": 0,
      "strengths": ["Good programming basics"],
      "weaknesses": ["Needs Spring Boot practice"],
      "recommendedSkills": ["Spring Boot"],
      "skillPriorities": {
        "Spring Boot": "High"
      },
      "summary": "Resume analysis summary"
    }
""".trimIndent()
```

```
return try {

    var responseText: String? = null
    var attempt = 0

    while (attempt < 1) {

        try {

            android.util.Log.d(
                "TalentTrack_AI",
                "Gemini attempt ${attempt + 1}"
            )

            val response = withTimeout(30000L) {

                withContext(Dispatchers.IO) {

                    client.models.generateContent(
                        "gemini-2.5-flash",
                        prompt,
                        null
                    )
                }
            }

            responseText = response.text()

            if (!responseText.isNullOrBlank()) {
                break
            }

        } catch (e: Exception) {

            val error = e.message.orEmpty()

            android.util.Log.e(
                "TalentTrack_AI",
                "Gemini attempt failed: $error, falling back to LocalAnalyzer"
            )

            return LocalAnalyzer.analyze(resumeText, jobDetails)
        }
    }

    val resultText =
        responseText ?: return LocalAnalyzer.analyze(resumeText, jobDetails)

    android.util.Log.d(
```

```
    "TalentTrack_AI",
    "Gemini response received"
)

val jsonStart =
    resultText.indexOf("{")

val jsonEnd =
    resultText.lastIndexOf("}")

if (
    jsonStart == -1 ||
    jsonEnd == -1
) {

    android.util.Log.e(
        "TalentTrack_AI",
        "Invalid JSON response: $resultText, falling back to LocalAnalyzer"
    )

    return LocalAnalyzer.analyze(resumeText, jobDetails)
}

val jsonString =
    resultText.substring(
        jsonStart,
        jsonEnd + 1
    )

val json =
    JSONObject(jsonString)
val matched =
    mutableListOf<String>()

val matchedArray =
    json.optJSONArray("matchedSkills")
        ?: json.optJSONArray("existingSkills")

if (matchedArray != null) {

    for (
    i in 0 until matchedArray.length()
    ) {

        matched.add(
            matchedArray.getString(i)
        )
    }
}
```

```
val missing =
    mutableListOf<String>()

val missingArray =
    json.optJSONArray("missingSkills")

if (missingArray != null) {

    for (
    i in 0 until missingArray.length()
    ) {

        missing.add(
            missingArray.getString(i)
        )
    }
}
```

APPENDIX – B
SCREENSHOTS

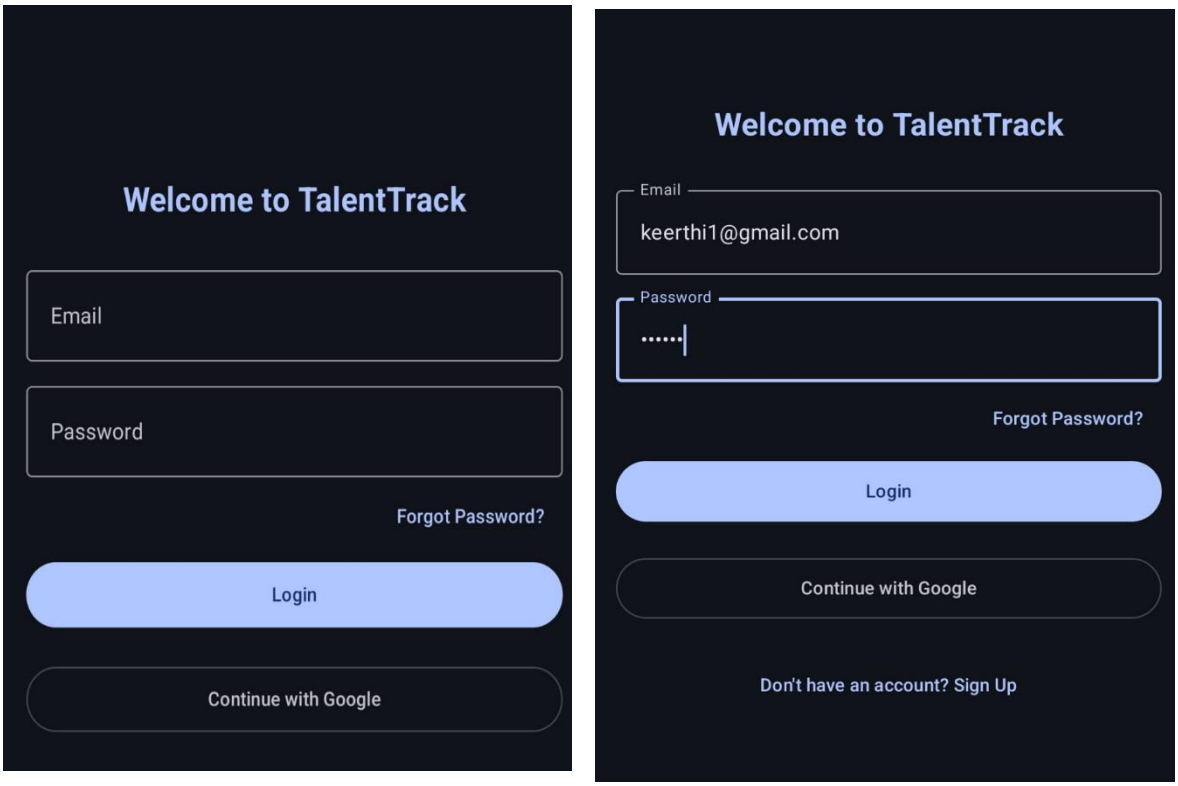


Figure B.1 Login and Registration Screen

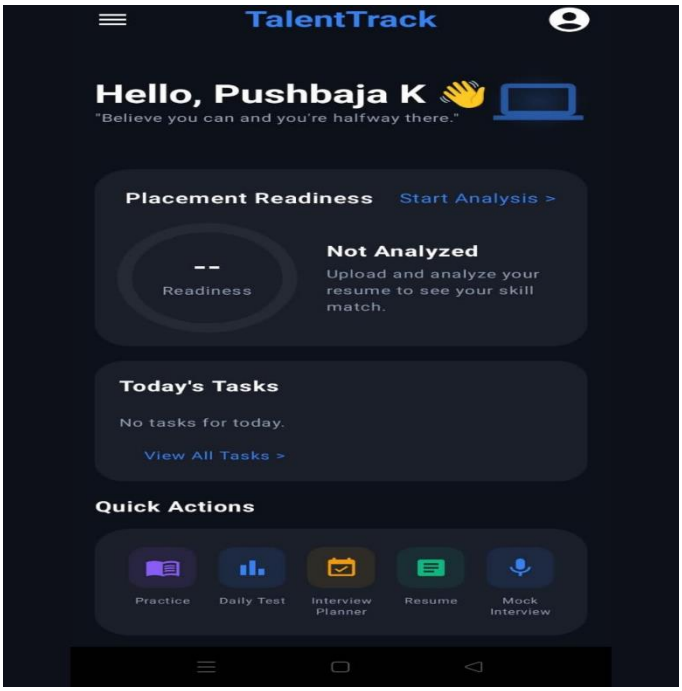


Figure B.2 TalentTrack Dashboard

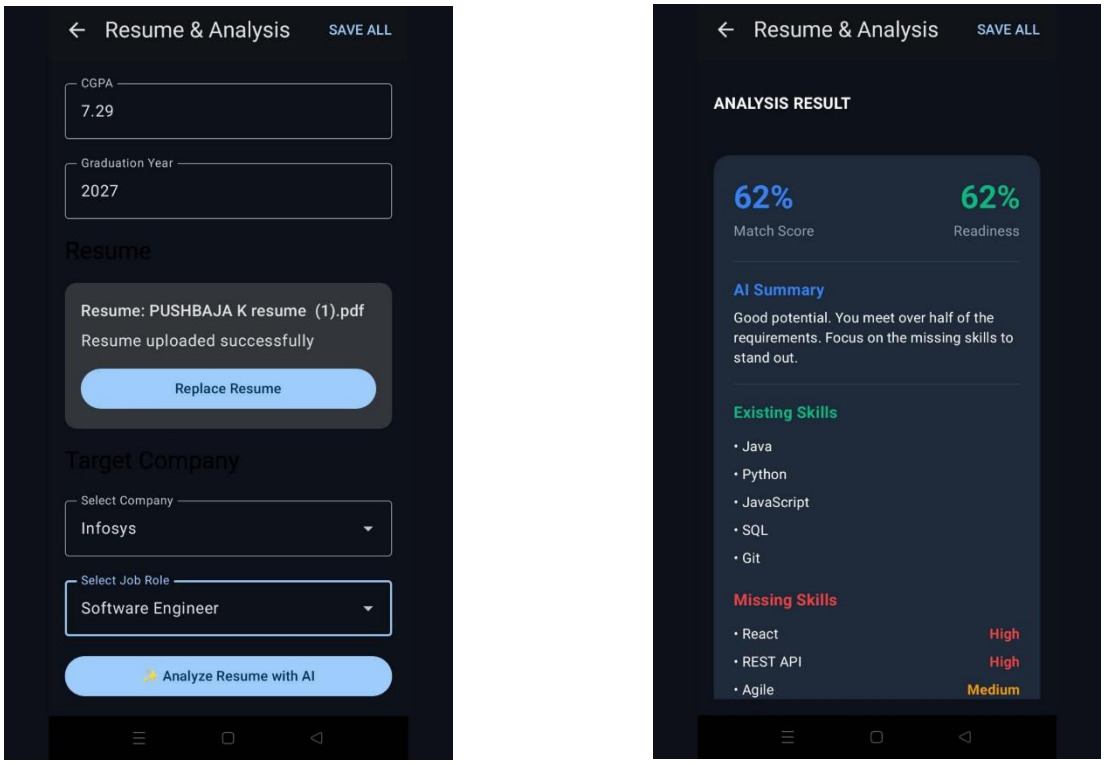


Figure B.3 Resume and Analysis Page

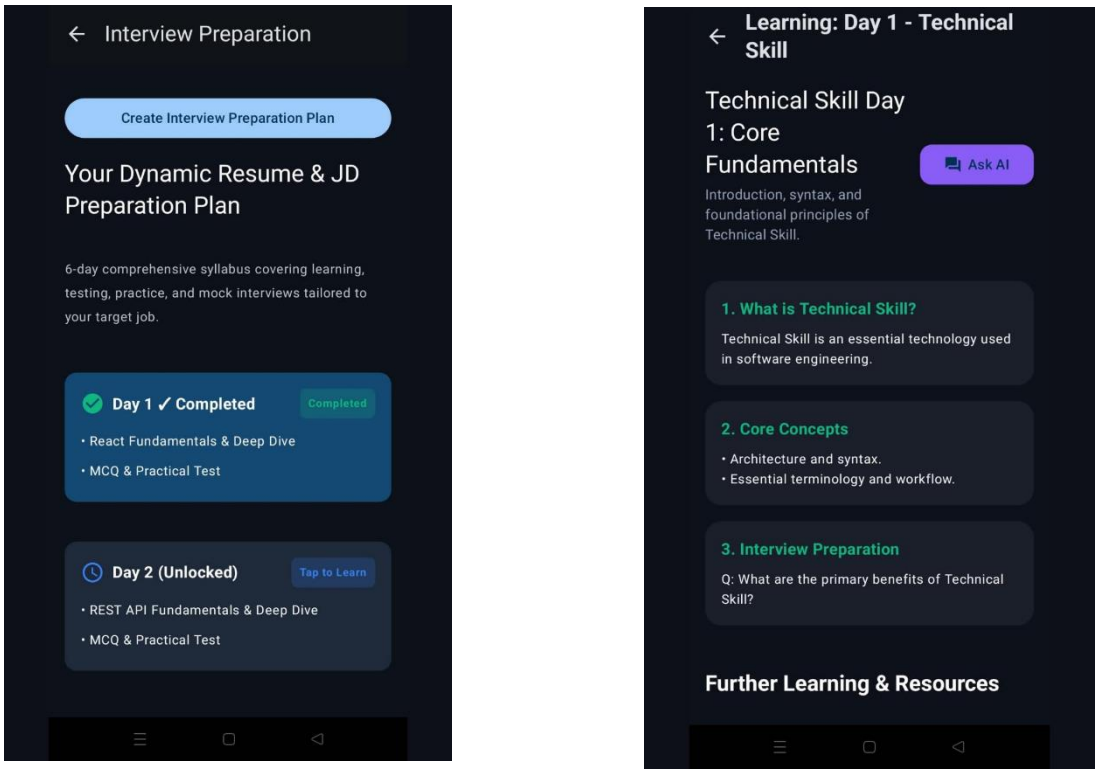


Figure B.4 Interview Preparation Page

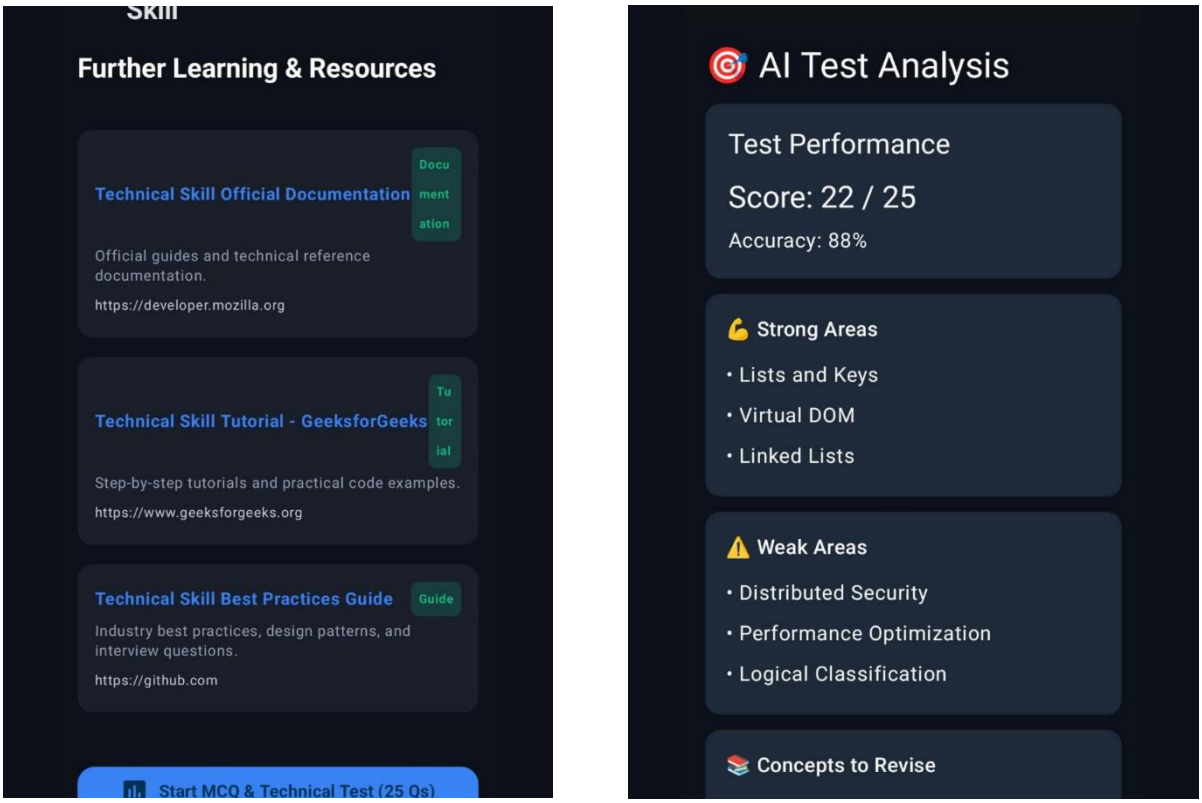


Figure B.5 Guidance And Test Analysis

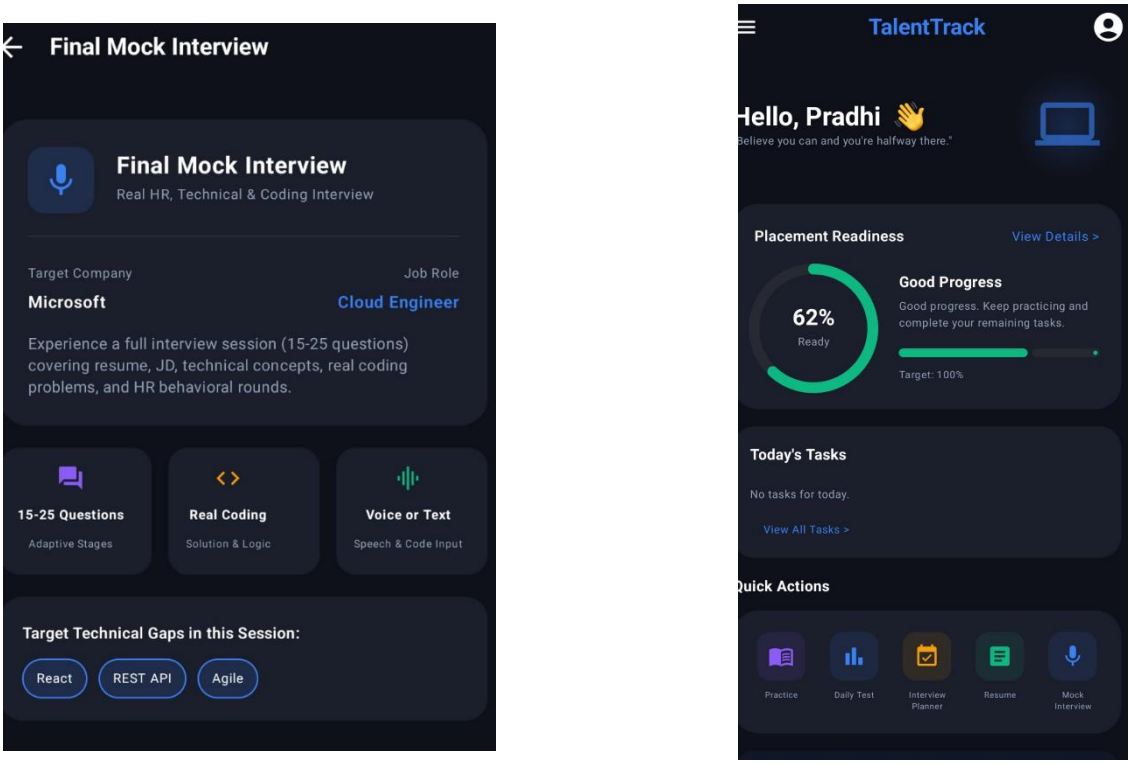


Figure B.6 Mock Interview And Readiness Engine

REFERENCES

1. Android Developers. Guide to app architecture: ViewModel, StateFlow and the data layer. developer.android.com.
2. Android Developers. Jetpack Compose documentation. developer.android.com/jetpack/compose.
3. Developing an Intelligent Resume Screening Tool With AI-Driven Analysis and Recommendation Features. (2025).
4. Goodfellow, I., Bengio, Y., and Courville, A. (2016). Deep Learning. MIT Press.
5. Kaya, M., & Bogers, T. (2026). Analyzing the Effects of a Human-in-the-Loop Candidate Recommendation Algorithm at Jobindex. ACM Transactions on Recommender Systems. https://doi.org/10.1145/3801092.
6. Kotlin Team. Kotlin Documentation. JetBrains. https://kotlinlang.org/docs/home.html.
7. Nayana, R., Sinchana, S. L., Shwetha, K., & Shreyas, L. (2025). Smart Job-Seeking Assistant and Web Scraping. International Journal of Computer Engineering in Research.
8. Nexus: A Multi-Modal Framework for Semantic Job Matching and AI-Driven Resume Analysis. (2026). Procedia Computer Science, 283, 922–935.
9. Real-Time AI-Based Skill Gap Analysis and Adaptive Career Guidance Using a Generative AI Framework for the Modern Job Market. (2025). 2025 5th Asian Conference on Innovation in Technology (ASIANCON). 76, 106–115.
10. Rojas-Galeano, S., et al. (2022). A Bibliometric Perspective on AI Research for Job-Résumé Matching. Computational Intelligence and Neuroscience, 2022, Article 8002363.